

Searching Binary Embeddings

Bitplanes and Prefix Backward Walk

13 September 2026

Abstract

I study three ways to search binary document vectors after selecting clusters: a full scan, Bitplanes, and Backward Walk. I compare their work, memory use and retrieval time using the same documents and queries. The experiments examine when the extra index work saves enough full document scores to reduce latency at a required recall.

1 Introduction

1.1 Exa method and end to end pipeline

Let's first understand the "original" pipeline that I am trying to improve upon.

Note that the architecture I am referring to here is based on the pipeline described in Exa's article [1]. I am aware that Exa's actual production system may have changed or improved since then, but for the purpose of this report, let's treat the architecture described in that article as the original baseline that we compare against.

Detail: In this pipeline, document embeddings are first shortened using Matryoshka representations, converted into binary signs, and assigned to clusters. At query time, the query is encoded, a subset of nearby clusters is selected, and the documents inside those clusters are scored using the floating-point query. The top candidates can then be reranked using the original uncompressed vectors.

The main part that I am trying to improve in this report is the search stage after cluster selection. The two new approaches introduced later, Bitplanes and Backward Walk, replace this step while keeping the rest of the pipeline largely the same.

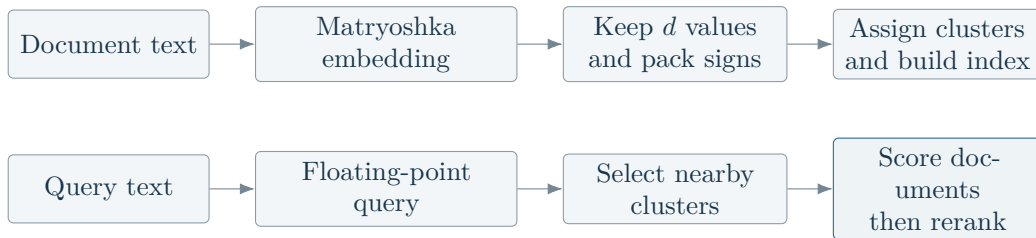


Figure 1: The baseline pipeline; B and C change document search inside the selected clusters.

1.2 Idea background

When I took a closer look at the pipeline, one part that particularly stood out to me was the comparison between the floating-point query and the binary-quantized document.

Consider a simple example:

$$q = [0.7, 0.5, -0.4, -0.6, 0.3]$$

and suppose a document is represented using binary signs, where 1 means +1 and 0 means -1. For example:

$$\text{Document} = 11000$$

The score is calculated by taking the dot product between the query values and the document signs. So for 11000, the score is

$$0.7(+1) + 0.5(+1) + (-0.4)(-1) + (-0.6)(-1) + 0.3(-1) = 1.9.$$

Now, something interesting follows from this. Just by looking at the signs of the query, we already know what the ideal binary document would look like.

For every positive query value, we would prefer the corresponding document bit to be 1, and for every negative query value, we would prefer it to be 0. Therefore, for the query above, the ideal binary pattern is

$$11001.$$

This document would achieve the maximum possible score:

$$0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5.$$

From here, we can think about every other document in terms of how far it is from this ideal pattern. More importantly, not every mismatch has the same cost. If we mismatch the first coordinate, where $|q_1| = 0.7$, the score decreases by 1.4. But if we mismatch the last coordinate, where $|q_5| = 0.3$, the score only decreases by 0.6.

In more formal terms, let

$$Q = \sum_i |q_i|$$

be the score of the ideal binary pattern. If a document b disagrees with the query sign at some coordinates, define its mismatch penalty as

$$p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|.$$

Then its score can be written as

$$S(q, b) = Q - 2p. \tag{1}$$

This observation is the main idea behind the approaches explored in this report. Instead of treating every binary document equally and scoring all of them one by one, can we use the query itself to guide the search toward documents that are closer to its ideal binary pattern, while avoiding unnecessary full-score calculations?

1.3 Exploring new methods

From the previous section, we know two useful things before we even start searching:

We know what the ideal binary document looks like from the signs of the query. We know that some bits matter more than others, based on the magnitude $|q_i|$.

The next question is therefore:

Can we build an index that uses this information directly, instead of treating every document inside the selected clusters equally?

This led me to explore two approaches.

The first, Bitplanes, starts from a large set of candidate documents and progressively narrows it down using the query.

The second, Backward Walk, approaches the problem from the opposite direction: it starts from a very specific binary pattern and progressively relaxes the constraints until enough useful candidates are found.

1.3.1 Bitplanes

Now that we know what the ideal document looks like based on the query, one idea is to build a data structure that makes filtering by individual bits very cheap.

Normally, binary documents are stored row by row:

ID 2: 10100
 ID 3: 11001
 ID 4: 11000

Instead, we can additionally store an inverted representation of the same bits:

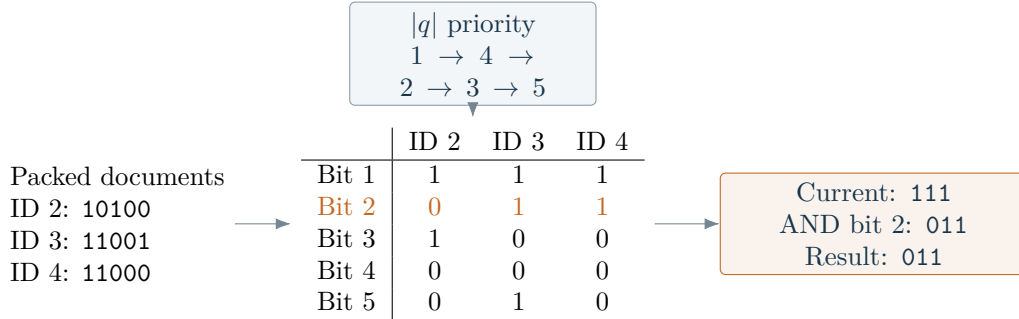


Figure 2: The same document signs, stored in an extra layout for bitwise filtering.

Each row above is what I call a bitplane: one bit position across many documents.

Suppose the query is

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

From the magnitude of the query values, we can prioritize the bits approximately as

$$1, 4, 2, 3, 5,$$

because

$$0.7 > 0.6 > 0.5 > 0.4 > 0.3.$$

Now suppose our current candidate mask is

$$111$$

meaning that IDs 2, 3 and 4 are all still possible candidates.

For bit 2, the query prefers 1. The corresponding bitplane is

$$011$$

so keeping the documents that match the query is simply

$$111 \& 011 = 011.$$

We immediately reduce the candidate set to IDs 3 and 4.

This is attractive because the filtering itself consists mostly of bitwise operations, which modern CPUs can perform very efficiently.

A simplified first version of the idea might therefore look like:

```

def full_score(query, code):
    return sum(q if bit else -q for q, bit in zip(query, code))

def greedy_search(query, cluster, k, leaf_size):
    order = sorted(range(len(query)), key=lambda i: -abs(query[i]))
    active = cluster.full_mask
    for bit in order:
        plane = cluster.planes[bit]
        preferred = plane if query[bit] >= 0 else ~plane
        active &= preferred
        if active.bit_count() <= leaf_size:
            break

    best = TopK(k)
    for row_id, code in cluster.rows_in_mask(active):
        best.add(row_id, full_score(query, code))
    return best.results()

```

However, this simple version has a major problem.

PROBLEM 1: Greedily following the preferred bits can miss the best document

Consider:

$$q = [0.6, 0.5, 0.4, 0.3].$$

The ideal binary document is 1111.

Suppose the database only contains:

Document A: 0111

Document B: 1000

If we greedily start with the first bit, the query prefers 1. We would therefore keep Document B and throw away Document A.

But their complete scores are:

$$S(A) = -0.6 + 0.5 + 0.4 + 0.3 = 0.6$$

while

$$S(B) = 0.6 - 0.5 - 0.4 - 0.3 = -0.6.$$

So Document A is actually better.

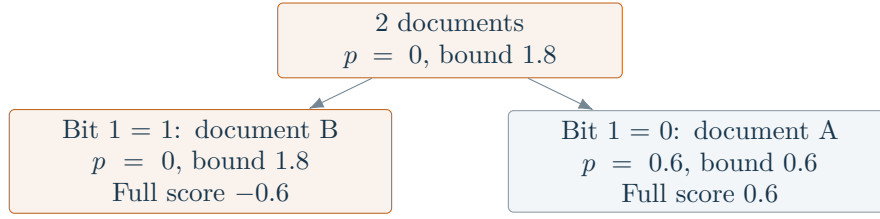
The problem is that matching one important bit does not guarantee that the remaining bits will also match.

Solution: branching

Instead of discarding the alternative, we keep both branches.

The branch matching the preferred bit receives no additional mismatch penalty. The alternative branch is still kept, but its penalty increases by $|q_i|$.

We then explore the branches in order of smallest accumulated penalty. This gives us a recursive search tree rather than one greedy path.



After scoring B, the other bound is $0.6 > -0.6$.
We must explore A; the first choice was not enough.

Figure 3: Keep the alternative until its score bound proves it cannot improve the result.

Stopping condition

Recall:

$$S(q, b) = Q - 2p, \quad Q = \sum_i |q_i|.$$

If a branch has already accumulated mismatch penalty p , any document below it must have penalty at least p .

Therefore,

$$U_{\text{branch}} = Q - 2p$$

is an upper bound on the score of every document inside that branch.

For example, suppose $Q = 2.5$ and we are looking for the top two results. We have already found 2.5 and 1.9, where 1.9 is the current top-2 threshold.

Now consider an unexplored branch with $p = 0.5$. Its absolute best possible score is

$$2.5 - 2(0.5) = 1.5.$$

Since $1.5 < 1.9$, nothing inside that branch can enter our current top two. We can safely discard the entire subtree.

So:

$$Q - 2p < \text{current } K\text{th score} \Rightarrow \text{prune branch}$$

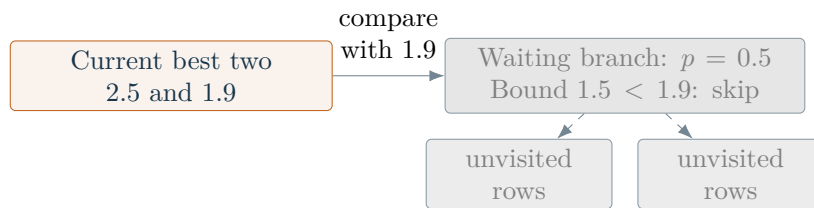


Figure 4: The whole gray subtree can be skipped because even its best possible score is too low.

If the best remaining branch in the queue already fails this test, then every other branch is worse and the entire search can stop.

The resulting algorithm is approximately:

```

from heapq import heappop, heappush
from itertools import count

def bitplane_search(query, clusters, k, leaf_size):
    order = sorted(range(len(query)), key=lambda i: -abs(query[i]))
    ideal_score = sum(abs(q) for q in query)
    sequence = count()
    queue = []
    for cluster in clusters:
        if cluster.full_mask:
            heappush(queue, (0, next(sequence), 0,
                             cluster.full_mask, cluster))

    best = TopK(k)
    while queue:
        penalty, _, depth, active, cluster = heappop(queue)
        upper_bound = ideal_score - 2 * penalty
        if best.full() and upper_bound < best.worst_score():
            break
        if active.bit_count() <= leaf_size or depth == len(query):
            for row_id, code in cluster.rows_in_mask(active):
                best.add(row_id, full_score(query, code))
            continue

        bit = order[depth]
        plane = cluster.planes[bit]
        preferred = plane if query[bit] >= 0 else ~plane
        matching = active & preferred
        # Keep the other half too; its remaining bits may be better.
        children = ((matching, 0), (active ^ matching, abs(query[bit])))
        for mask, extra_penalty in children:
            if mask:
                heappush(queue, (penalty + extra_penalty,
                                 next(sequence), depth + 1, mask, cluster))

    return best.results()

```

This is essentially the branch-and-bound Bitplane search used in the current experiments.

PROBLEM 2: Bitwise operations are cheap, but the bitmap itself can still be large

There is another issue.

A bitwise AND sounds almost free, but it is not literally one operation when the cluster contains thousands or millions of documents.

Suppose one cluster contains 6400 documents. With 64-bit machine words, one bitplane contains

$$6400/64 = 100$$

machine words.

So one split roughly requires us to loop over those 100 words.

Even worse, suppose after several splits only 80 documents remain. The physical mask can still be 100 words wide. So the next split may still need to process those same 100 words.

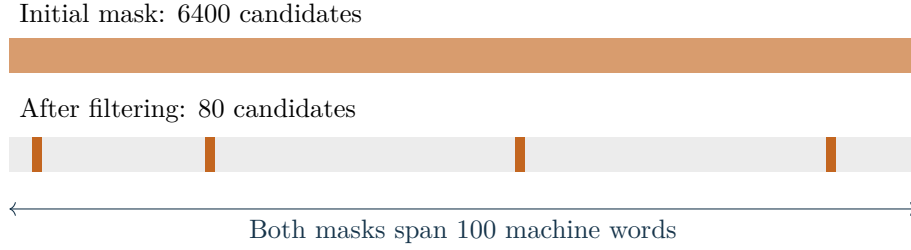


Figure 5: Fewer active document positions do not make this dense mask physically shorter.

Bitmap traversal cost. In general, for a cluster containing n documents, one dense bitplane operation costs roughly $O(\lceil n/64 \rceil)$ word operations. If we visit J split nodes, this becomes approximately $O(J \lceil n/64 \rceil)$, before even counting queue operations and the final document scoring. This was one of the important findings from the earlier experiments: reducing the number of fully scored documents does not automatically mean reducing total query time, because the bitmap traversal itself can become expensive.

Solution 1: Parameter tuning. This problem is technically something we can optimize. For example, we can tune cluster size during coarse routing; number of clusters probed; how deep we split; leaf size; when we switch back to ordinary full scoring. So Bitplanes is not fundamentally broken. The question is finding the point where scores avoided are worth more than additional bitmap traversal.

Solution 2: Other approach. However, thinking about this cost led me to a second idea. Bitplanes starts with a large set of documents and repeatedly narrows it down. What if we instead did the opposite? If Bitplanes can be thought of as bottom-up, then Backward Walk can be thought of as top-down: start from the most specific binary pattern we can describe and progressively broaden the search only when necessary.

1.3.2 Backward Walk

Backward Walk builds on the same observation:

Given a query, we already know what its ideal binary document should look like.

However, instead of starting with thousands of documents and filtering them using bitplanes, we try to start directly from a small bucket of documents that already looks similar to the query.

Suppose the binary query begins with 101101001011....

Using all 256 bits as a lookup key would be far too specific. With a large binary space, the probability that an exact 256-bit pattern actually exists in the database is extremely small.

Instead, suppose we choose a prefix length x . For example, the first 16 bits:

1011010010110101

During index construction, documents sharing this prefix are placed in the same bucket or contiguous range.

At query time, we directly look up this prefix. If it contains enough useful documents, we score them using the complete 256-bit score. If it is empty, or contains too few candidates, we remove one constraint:

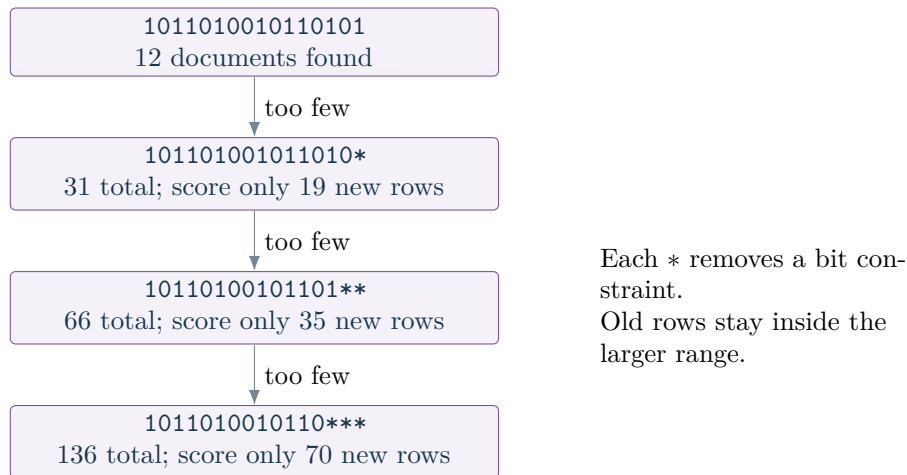


Figure 6: Illustrative counts: $12 + 19 + 35 + 70 = 136$ distinct rows scored, not $12 + 31 + 66 + 136$.

Every step moves to a larger candidate set.

This is why I call the method Backward Walk: instead of progressively adding constraints to a large candidate set, we begin from a very specific location and progressively walk backward toward more general prefixes.

Note that this bit prefix already has an exponential property: if the bits are independent and equally likely to be 0 or 1, each removed bit doubles the expected number of matching documents.

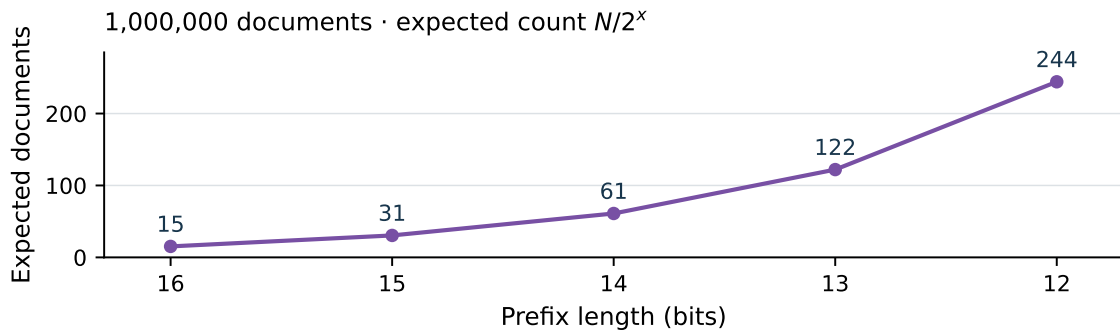


Figure 7: Expected counts for one global cluster; labels are rounded. This estimates rows, not recall.

With a trie or documents ordered by their prefix, the previous range is contained inside the parent range. We only need to process the newly introduced documents.

A simplified query algorithm becomes:


```

def backward_walk(query, clusters, k, start_depth, candidate_target):
    signs = tuple(q >= 0 for q in query)
    previous = [None] * len(clusters)
    best = TopK(k)
    scored = 0
    for depth in range(start_depth, -1, -1):
        for index, cluster in enumerate(clusters):
            left, right = cluster.prefix_range(signs[:depth])
            if previous[index] is None:
                new_ranges = [(left, right)]
            else:
                old_left, old_right = previous[index]
                # Only the new rows beside the old range need scores.
                new_ranges = [(left, old_left), (old_right, right)]
            for start, stop in new_ranges:
                for row_id, code in cluster.rows_in_range(start, stop):
                    best.add(row_id, full_score(query, code))
                scored += stop - start
            previous[index] = (left, right)

        # Finish this depth in every cluster before testing the target.
        if candidate_target > 0 and scored >= candidate_target:
            break
    return best.results()

```

There is one important distinction here.

Stopping simply because we have collected, for example, 100 candidates is an approximate stopping rule. It does not mathematically prove that a better document does not exist outside the current prefix.

Therefore, in the experiments, recall still needs to be measured against the same exact reference results. An exact version would require a valid upper bound on the scores of documents outside the currently visited prefix.

Relationship to HNSW

At first glance, this approach may sound somewhat similar to HNSW because both methods use a hierarchy to progressively change the search space.

However, the hierarchy is being used differently.

HNSW builds a proximity graph. During a query, it begins from sparse upper layers, navigates toward promising neighbors, and progressively descends toward the denser bottom layer [2].

Backward Walk does not need to navigate between neighboring document vectors. The query itself directly tells us where to begin:

query → binary prefix → exact prefix lookup

We then broaden that prefix only when necessary.

So conceptually:

	HNSW	Backward Walk
Index	Proximity graph	Prefix / key hierarchy
Query start	Sparse graph layer	Query's own binary prefix
Movement	Neighbor → neighbor	Specific prefix → broader prefix
Index insertion	Find neighbors + maintain edges	Compute prefix + insert into bucket/range
Search signal	Vector distance	Prefix agreement

Potential advantage 1: cheaper index construction and updates

One possible advantage that I would like to investigate further is the cost of adding new documents. For HNSW, inserting a new vector generally requires searching the existing graph to find suitable neighbors and then creating or updating graph connections.

For Backward Walk, the conceptual insertion process is much simpler:

document $\rightarrow$ embedding $\rightarrow$ binary signs $\rightarrow$ prefix key $\rightarrow$ corresponding bucket

There is no nearest-neighbor graph that needs to be constructed for each new document. So Backward Walk may be attractive for a search index that changes frequently.

I would frame this as a hypothesis to verify, because the actual insertion cost still depends on how the buckets, ranges and on-disk/in-memory index are implemented.

Potential advantage 2: Matryoshka prefixes give the relaxation some meaning

There is also a useful connection with Matryoshka embeddings.

A Matryoshka model is trained so that shorter prefixes of the embedding remain useful representations [3].

Therefore, moving from, for example, 16 bits $\rightarrow$ 15 bits $\rightarrow$ 14 bits is not simply dropping arbitrary dimensions from an ordinary embedding. We are progressively moving to shorter prefixes of a representation that was specifically trained to retain useful information at multiple prefix lengths.

However, this does not mean that the first coordinate is always more important than the second, or that the nearest neighbor under 16 dimensions must remain the nearest neighbor under 15 dimensions.

Matryoshka training gives us a reason to believe that shorter prefixes remain meaningful; the actual recall of each prefix still has to be measured.

2 Preliminary calculations

2.1 Time complexity

Here N counts documents, d bits per document, and K retained results. The index has C clusters; a query opens P of them. We call the opened set $\mathcal{O}$, with n_c rows in cluster c . Its mask occupies $W_c = \lceil n_c/64 \rceil$ words, rounded up because a partly used word still takes eight bytes. The Experiment section declares the measured data and settings.

One example for all three methods. Open one cluster containing these three rows. Return the best two results: $N = 3$, $C = P = 1$, $d = 5$, $K = 2$.

$$q = [0.7, 0.5, -0.4, -0.6, 0.3], \quad \text{ideal signs: } 11001.$$

ID	Document bits	Full score	Calculation
0	10100	0.1	$0.7 - 0.5 - 0.4 + 0.6 - 0.3$
1	11001	2.5	$0.7 + 0.5 + 0.4 + 0.6 + 0.3$
2	11000	1.9	$0.7 + 0.5 + 0.4 + 0.6 - 0.3$

Bits are shown in their original position order, left to right. The displayed scores are rounded. We use F_A , F_B and F_C for the number of full document scores calculated by A, B and C respectively.

2.1.1 Original method

Step 1: open the cluster and prepare the query.

Open cluster 0: [ID 0, ID 1, ID 2]. One cluster lookup here. Following Exa's four-sign table method [1], split the five query values into [0.7, 0.5, -0.4, -0.6] and [0.3, 0, 0, 0]. Each group has 16 sign patterns: $2 \times 16 = 32$ table entries. The final group is padded.

The code checks four slots per entry: $32 \times 4 = 128$ slot visits. Table setup is $O(d)$ for a fixed group width of four. We call the cluster-selection time T_{route} .

Step 2: score each document.

$$\text{ID 0} \longrightarrow 0.1 \quad \text{ID 1} \longrightarrow 2.5 \quad \text{ID 2} \longrightarrow 1.9$$

Loop over three rows; each reads two score-table entries: $3 \times 2 = 6$ terms. For F_A rows, this is $F_A \lceil d/4 \rceil$ terms, or $O(F_A d)$ work.

Step 3: keep the best two.

$$[0:0.1] \longrightarrow [1:2.5, 0:0.1] \longrightarrow \boxed{[1:2.5, 2:1.9]}.$$

We offer three scores and keep at most two. The display above is sorted for readability. Updating the retained results costs up to $O(F_A \log(K+1))$; putting the final results in order costs $O(K \log(K+1))$. Writing $K+1$ also covers the constant work of keeping just one result.

Total. One cluster lookup, 32 table entries built, six table terms read and three scores offered. These are different kinds of work, so we do not add them as equal CPU operations.

$$F_A = \sum_{c \in \mathcal{O}} n_c, \quad \boxed{T_A \approx T_0 + F_A a_A.}$$

T_0 includes cluster selection, cluster lookups, table setup and final sorting. a_A is average time per full score and result update. For roughly equal clusters, $n \approx N/C$ and $F_A \approx Pn = PN/C$. With C , P , d and K fixed, scan work then grows in proportion to N . We measure the time coefficients for the machine and settings being used. Reading a larger index from RAM can cost more than reading a small one from the CPU cache, so one fitted coefficient need not work at every size.

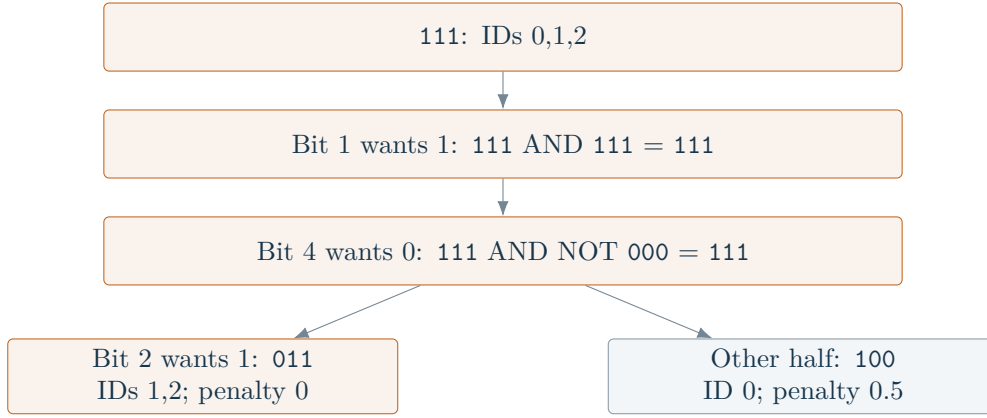
2.1.2 Bitplanes

Use the same query and rows. Stop splitting when at most two rows remain, keeping other work limits unlimited.

Step 1: open, prepare and order the bits.

We open the same cluster and build the same 32 score-table entries as A. Sorting the five query positions by absolute value gives 1 (0.7) $\rightarrow$ 4 (0.6) $\rightarrow$ 2 (0.5) $\rightarrow$ 3 (0.4) $\rightarrow$ 5 (0.3). This costs $O(d \log d)$. One pass adds the five absolute values to obtain 2.5, costing $O(d)$. We then write one word for mask 111, where all rows are active. Across the opened clusters, the initial masks require $\sum_{c \in \mathcal{O}} W_c$ word writes.

Step 2: split the mask.



The first two splits remove nothing. The third separates ID 0 from IDs 1 and 2. Every mask still has three positions and occupies one 64-bit word, so the three splits visit $3 \times 1 = 3$ word positions. If J_c is the number of splits in cluster c , the total is $\sum_{c \in \mathcal{O}} J_c W_c$ split-word visits. Each visit reads a mask and a bitplane, forms and writes both children, and counts set bits in the matching child. It is not a single CPU instruction.

Step 3: score the two remaining rows.

Mask 011 fits the leaf limit. We read one mask word, then score IDs 1 and 2. Two scores use $2 \times 2 = 4$ table terms and leave $[1: 2.5, 2: 1.9]$ as the best results. If H_c is the number of final masks read in cluster c , those reads visit $\sum_{c \in \mathcal{O}} H_c W_c$ words. Full scoring costs $O(F_B d)$, with up to $O(F_B \log(K + 1))$ work for result updates.

Step 4: check the waiting branch and stop.

Its best possible score is $2.5 - 2(0.5) = 1.5$, below the retained score 1.9, so we skip ID 0. This bound check is $O(1)$ once the query sum and penalty are known. Let V count processed candidate sets. Here $V = 4$: three splits and one leaf. Including the P initial sets, queue work is bounded by $O((P + V) \log(P + V + 1))$.

Total. Same table setup as A; sum five weights; sort five positions; write one initial mask; visit three split words and one final-mask word; score two rows; skip one row. Fewer scores alone do not prove lower time.

$$T_B \approx T_0 + b \sum_{c \in \mathcal{O}} J_c W_c + F_B a_B + T_{\text{other}, B}.$$

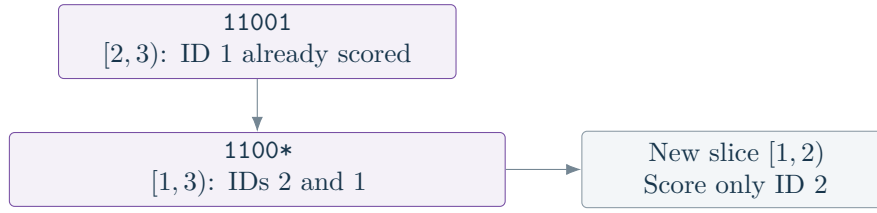
b is time per split-word visit, and a_B time per full score and result update. $T_{\text{other}, B}$ covers query ordering and weight sums, initial masks, queues and final-mask reads. For roughly equal clusters, let S be average splits per cluster and $f_B = F_B / F_A$ the scored fraction. The split and score terms become $PS \lceil N / (64C) \rceil b$ and $f_B (PN / C) a_B$. Here $S = 3$ and $f_B = 2/3$.

2.1.3 Backward Walk: prefix relaxation

Let h be stored prefix width, x starting depth and ℓ final depth. With the same query and rows, set $h = x = 5$ and a candidate target of two. We check that target after completing each depth across all selected clusters.

Step 1: find the full prefix.

Before queries arrive, the index stores the rows in prefix order: 10100, 11000, 11001. We build the common score table and read h query signs once to form key 11001, taking $O(h)$ work. Later depths reuse that key. Two binary searches find prefix range $[2, 3)$, containing ID 1. Its full score is 2.5. With only one candidate scored, we continue.

Step 2: remove the last constraint.

The parent contains the old range, so we score only the new row, ID 2, obtaining 1.9. Two distinct candidates have now been scored. We stop and return IDs 1 and 2, recovering both reference IDs here. The same candidate count need not recover both on other data.

Step 3: count the work.

Two nonzero depths in one cluster make four binary-search calls. Each call uses at most two key comparisons here, giving at most eight comparisons, plus range checks. The two scores read four table terms. Table setup, query-key construction, result updates and final sorting also take time.

There are $L = x - \ell + 1$ visited depths. Of these, L_+ are nonzero: $L_+ = L$ if $\ell > 0$, and $L_+ = L - 1$ if $\ell = 0$. Depth zero uses the known whole-cluster range without binary search. The exact boundary-search count is therefore $2PL_+$.

$$T_C \approx T_0 + T_{\text{key}} + 2L_+ t_b \sum_{c \in \mathcal{O}} \log_2(n_c + 1) + F_C a_C + T_{\text{ranges}}.$$

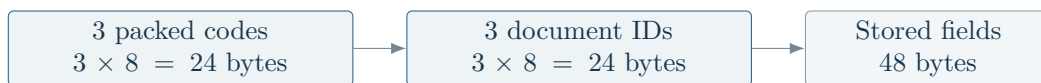
T_{key} builds the query key once, t_b estimates time per key comparison, and a_C covers a full score and result update. T_{ranges} counts prefix and new-slice checks at all L depths. With roughly equal sizes $n \approx N/C$, the binary-search term becomes $2PL_+ \log_2(n + 1)t_b$.

Range bookkeeping is $O(PL)$, scoring $O(F_C d)$, and result updates up to $O(F_C \log(K + 1))$. Under the default stopping rule, a zero candidate target reaches depth zero and scores all opened rows once. Section 3.3 adds an optional score check that can prove the answer sooner; its setup and checks are included in T_{key} and T_{ranges} . We need measured coefficients to turn these counts into milliseconds.

2.2 Memory complexity

We keep N documents of d bits in C clusters, opening P per query. Cluster c has n_c rows and $W_c = \lceil n_c/64 \rceil$ words per mask. In bytes, a packed word or ID uses eight, a float query value four, and a double-precision score eight. Subscripts on M and Q name the stored and query fields being counted.

For the example, $N = 3$, $d = 5$, $C = P = 1$ and $K = 2$. We count named fields first; unused capacity, overlapping buffers and the rest of the process need separate measurements.

2.2.1 Original method**Step 1: store the codes and IDs before search.**

Each five-bit row uses one eight-byte word: $M_{\text{rows}} = N(8 \lceil d/64 \rceil + 8)$ bytes, or $O(N \lceil d/64 \rceil + N)$. Routing adds M_{routing} bytes. If the router reads d_R coordinates, its C float32 centroids and eight-byte labels use $4Cd_R + 8C$ bytes. The one-cluster example needs no centroid search.

Step 2: hold the query, score table and current results during search.

$$\boxed{5 \times 4 = 20} \quad + \quad \boxed{32 \times 8 = 256} \quad + \quad \boxed{2 \times (8 + 8) = 32} \quad = \quad 308 \text{ bytes.}$$

Left to right are the query, score table and one buffer of result IDs and scores. Their field count is $Q_0 = 4d + 128 \lceil d/4 \rceil + 16K$, or $O(d + K)$. The heap, final sorting and Python output can overlap in memory; Q_0 counts one result buffer, not the complete peak.

Total counted fields: 48 bytes stored + 308 query bytes = 356 bytes. This is not total process RAM; cluster records and allocation overhead remain additional.

2.2.2 Bitplanes

Step 1: keep A's rows and add the bitplanes.

	ID 0	ID 1	ID 2	Packed storage
Bit 1	1	1	1	8 bytes
Bit 2	0	1	1	8 bytes
Bit 3	1	0	0	8 bytes
Bit 4	0	0	0	8 bytes
Bit 5	0	1	0	8 bytes

Each plane has three useful bits but occupies one word. Five planes add $5 \times 8 = 40$ bytes. Stored fields: $48 + 40 = 88$ bytes. Formula: $M_{\text{planes}} = 8d \sum_c \lceil n_c/64 \rceil$ bytes. Include padding in every cluster.

Step 2: hold masks while splitting.

$$\underbrace{111}_{\text{parent: 8 bytes}} \longrightarrow \underbrace{011}_{\text{child: 8 bytes}} + \underbrace{100}_{\text{child: 8 bytes}}.$$

The parent and both full-width children coexist: $3 \times 8 = 24$ live mask bytes. Five sorted positions add $5 \times 8 = 40$ bytes on a 64-bit machine, giving $308 + 24 + 40 = 372$ bytes for these query fields. Queue records are extra. Let $A_c(t)$ count live masks in cluster c at time t , including waiting branches and the parent being split. Their peak storage is $M_{\text{masks}} = 8 \max_t \sum_{c \in \mathcal{O}} A_c(t) W_c$.

Total counted fields: 88 bytes stored + 372 query bytes = 460 bytes. Stored size is $O(N \lceil d/64 \rceil + N + d \sum_c W_c)$; query masks grow with the number and width of live branches, not just the number of remaining document IDs.

2.2.3 Backward Walk

Step 1: store rows in prefix order.

Position	Full binary document	ID
0	10100	0
1	11000	2
2	11001	1

The full codes and IDs use $3 \times (8 + 8) = 48$ bytes. Three four-byte prefix keys add 12 bytes, totaling 60. The key type supports stored widths up to 32 bits. Every prefix range uses this same sorted array, without a separate row list per depth.

$$\boxed{M_{C,\text{fields}} = N (8 \lceil d/64 \rceil + 8 + 4) + M_{\text{routing}}.}$$

This counts the fields used by the index. Its allocated arrays can be larger because capacity is rounded up, and construction can hold temporary copies alongside the finished arrays.

Step 2: hold range boundaries during search.

Each selected-cluster record contains an eight-byte cluster pointer and two eight-byte offsets for its previous range: $(8 + 8 + 8)P = 24P$ bytes on a 64-bit machine. The boundaries alone account for $16P$. The query key uses four more bytes, so the example adds $24 + 4 = 28$ bytes to A's common query fields.

The previous range tells us which rows were already scored, so we need neither a bitmap over all documents nor a list of visited IDs.

Total counted fields: 60 bytes stored plus $308 + 24 + 4 = 336$ query bytes gives 396 bytes for this example, before capacity, local variables and overlapping buffers. Persistent space is $O(N \lceil d/64 \rceil + N)$ plus routing, and query space is $O(P + d + K)$. The Experiment section reports process memory separately, including the peak during construction.

2.3 Recall estimation

If the reference IDs are $[1, 2]$ and we return $[1, 0]$, we recover one of the two required IDs: recall is $1/2 = 50\%$. Returning $[1, 2]$, as all three worked searches above do, gives $2/2 = 100\%$. We find the reference by scoring every document with the full binary score and using the same document-ID rule to resolve ties.

Let K be the number of reference results per query and n_q the number of queries. For query q , G_q is the set of reference IDs and $O_{a,q}$ is the set returned by method a , where a is A, B or C. The intersection contains the IDs shared by both sets, and the bars count them. Across the full cached query set, average recall is

$$\bar{R}_a = \frac{1}{n_q K} \sum_q |G_q \cap O_{a,q}|.$$

2.3.1 Original method

Let g_q count reference IDs in the opened clusters. A scores every opened row using the reference score and tie order, so all g_q of those IDs remain in its local top K . Its recall is g_q/K .

2.3.2 Bitplanes

Suppose eight of ten reference IDs are in the opened clusters, and B returns six of those eight. Recall is $(8/10)(6/8) = 60\%$. We call the fraction recovered within the opened clusters $r_{B,q}$, giving recall $(g_q/K)r_{B,q}$. If $g_q = 0$, recall is zero. Unlimited local search with safe bounds recovers every opened reference ID.

2.3.3 Backward Walk

Let $D_{q,\ell}$ be the rows in the opened clusters matching the query's first ℓ bits. After completing depth ℓ across those clusters, every row in this set has been scored once. Keeping its full-score top K gives recall $|G_q \cap D_{q,\ell}|/K$. We need the intersection with the reference IDs to calculate that fraction; knowing how many rows match the prefix does not tell us how many reference results they contain.

Consider $q = [0.1, 0.9]$ and two documents, 10 and 01. The ideal pattern is 11, but it is absent. If we shorten it to $1*$, we find 10, whose score is $0.1 - 0.9 = -0.8$. The other document, 01, scores $-0.1 + 0.9 = 0.8$ and is actually better. Stopping after finding one candidate would therefore give zero top-one recall. We must widen to depth zero to reach that other document. The fixed

prefix order removed the more important second bit first; Matryoshka training does not sort each query’s values by magnitude.

At depth zero, $D_{q,0}$ contains every opened row, so C matches A’s recall for the same routing choice. With the default stopping rule, a zero candidate target reaches this depth. The score check in Section 3.3 can prove the same result sooner. A positive target can still stop earlier and miss reference IDs, because a candidate count does not bound the scores of unseen documents. Opening every cluster and completing exact local search gives an exact search over the whole collection.

A simple approximation. Suppose 80% of the reference documents agree with the query’s first sign, and 75% agree with its second sign. If the two agreements are independent, about $0.8 \times 0.75 = 60\%$ agree with both. The multiplication is an assumption about those events; it does not follow from the length of the embedding.

For a fixed ending depth ℓ , let p_i be the fraction of opened reference documents that agree at position i , counted across queries. Let $\bar{R}_A$ be the fraction of all reference IDs whose clusters we opened. Under independent sign agreement, we would estimate

$$\hat{R}_C(\ell) = \bar{R}_A \prod_{i=1}^{\ell} p_i.$$

The hat marks an estimate. We can obtain the p_i values from a sample of queries and their exact reference documents, then compare this product with the fraction that actually matches the whole prefix. For one global cluster, $\bar{R}_A = 1$. For clustered search, the rates must describe reference documents in the opened clusters.

This calculation assumes every query ends at the same depth. In the actual walk, the candidate target decides when each query stops. We therefore check reference membership at each query’s actual ending prefix. Substituting the average ending depth into the product would ignore how document counts and sign matches vary together. The experiment checks the simpler fixed-depth approximation separately.

2.4 Summary

Time complexity

Method	Approximate query time
Original method	$T_0 + a_A \sum_{c \in \mathcal{O}} n_c$
Bitplanes	$T_0 + b \sum_{c \in \mathcal{O}} J_c W_c + F_B a_B + T_{\text{other,B}}$
Backward Walk	$T_0 + T_{\text{key}} + 2L_+ t_b \sum_{c \in \mathcal{O}} \log_2(n_c + 1) + F_C a_C + T_{\text{ranges}}$

Here $\mathcal{O}$ is the opened set, n_c its cluster sizes, W_c mask widths and J_c split counts. F_B and F_C count scores; L_+ counts nonzero prefix depths. The a coefficients give time per score and result update, b per split-word visit, and t_b per key comparison. T_0 includes routing, common setup and final sorting; T_{key} builds the key and any optional query-weight summaries, T_{ranges} checks ranges, and $T_{\text{other,B}}$ covers B’s ordering, masks and queues.

Memory complexity

Method	Stored fields	Query fields
Original method	M_0	Q_0
Bitplanes	$M_0 + M_{\text{planes}}$	$Q_0 + Q_B$
Backward Walk	$M_0 + 4N$	$Q_0 + 24P + 4 + Q_{\text{exact}}$

$M_0 = N(8 \lceil d/64 \rceil + 8) + M_{\text{routing}}$ counts full codes, IDs and routing fields. $M_{\text{planes}} = 8d \sum_c W_c$ includes every cluster’s bitplanes. $Q_0 = 4d + 128 \lceil d/4 \rceil + 16K$ counts the query, table and one result buffer; Q_B adds bit ordering, live masks and queue records. P is the number of opened

clusters. $Q_{\text{exact}} = 8x + 8$ counts C's prefix minima and query sum when the optional score check is used at positive starting depth x ; it is zero otherwise. These field counts are separate from allocated array capacity and process RAM.

Recall estimation

Method	Per-query recall	Local exactness condition
Original method	g_q/K	Score all opened documents
Bitplanes	$(g_q/K)r_{B,q}$	Complete search with safe bounds
Backward Walk	$ G_q \cap D_{q,\ell} /K$	Reach depth zero, or prove the score bound

K is the result count; g_q counts opened reference IDs; $r_{B,q}$ is B's recovered fraction of those IDs. G_q is the reference set, and $D_{q,\ell}$ is C's visited prefix range across the opened clusters. The batch value divides total recovered IDs by $n_q K$, using all n_q cached queries. It measures agreement with the full binary ranking.

3 Deriving Optima

For each prepared pool and result count K , we want the lowest whole-query time that meets the recall and RAM limits. The Experiment section declares the unchanged inputs, parameter grid and limits.

3.1 Original method

A reference document in the router's fifth cluster is scored when we open $P \geq 5$ clusters. Let C be total clusters, n_q the query count, K the result count and ρ the required average recall. Write $s_{qi}^{(C)}$ for the cluster rank of reference result i for query q . The smallest probe count meeting the target is

$$P_A(C, \rho) = \min \left\{ P : \frac{1}{n_q K} \sum_{q=1}^{n_q} \sum_{i=1}^K \mathbf{1}[s_{qi}^{(C)} \leq P] \geq \rho \right\}.$$

The indicator $\mathbf{1}$ counts an ID when its cluster is open. We increase the cutoff if it splits tied centroid scores, repeating until no query has a tie across the boundary. Every method receives the same probe choices for a comparison. A one-cluster scan provides an exhaustive reference.

These ranks require actual queries, centroids and document assignments. Document and cluster counts alone cannot determine the cutoff. We calculate it separately for each declared K using all cached queries.

For a conditional derivative, suppose clusters are equal and P stays fixed. Let α be routing time per centroid, β time per full score and result update, and T_{fixed} the setup and final sorting time held constant here. With N documents,

$$T_A(C) \approx T_{\text{fixed}} + \alpha C + \beta NP/C, \quad \frac{dT_A}{dC} = \alpha - \frac{\beta NP}{C^2}, \quad C^* = \sqrt{\frac{\beta NP}{\alpha}}.$$

Setting the derivative to zero gives C^* . At fixed recall, however, $P_A(C, \rho)$ can change with C , so this is only a conditional balance. We compare allowed cluster counts using actual sizes and probe choices.

3.2 Bitplanes

A 6400-document mask occupies 100 words. Avoiding 100 of the remaining scores saves 100 full-score calculations but adds 100 split-word visits. For n documents, let $W = \lceil n/64 \rceil$ be mask width, ΔF scores avoided, a_B time per score and result update, and b time per split-word visit. With added queue and final-mask time $\Delta T_{\text{other,B}}$, splitting helps when

$$\Delta F a_B > Wb + \Delta T_{\text{other,B}}.$$

The saved time must exceed the added work while preserving the required recall. Suppose every cut halves one followed set and other branch costs stay fixed. After j cuts, about $n2^{-j}$ rows remain, giving

$$T_B(j) \approx T_0 + jWb + n2^{-j}a_B + T_{\text{other,B}}, \quad j^* = \log_2 \left(\frac{n \ln(2)a_B}{Wb} \right).$$

Here T_0 is routing, common setup and final sorting time. Differentiating gives the candidate depth j^* ; $\ln$ denotes the natural logarithm. Real cuts may remove no rows, and waiting branches add work, so we test the declared leaf sizes and node budgets rather than treating j^* as a recall guarantee.

A node budget counts visited sets, including leaves. A leaf limit at least as large as a cluster lets us score it immediately. B receives all of A's routing choices, but its local limits can lose additional reference IDs. We therefore record the work counts and check the returned IDs.

3.3 Backward Walk

Alongside C and P , we choose starting depth x and candidate target M . With h stored key bits, $0 \leq x \leq h \leq \min(d, 32)$; full scores use all d document bits. A deeper start selects fewer or the same rows, but may need more widening steps.

Let $F_A = \sum_{c \in \mathcal{O}} n_c$ count opened rows. Under independent, equally likely signs, a fixed prefix of depth x would contain $F_A/2^x$ rows on average. For $M > 0$, matching this count to M gives $x \approx \log_2(F_A/M)$, with $F_A \approx PN/C$ if clusters are equal. That is a row-count estimate, not a prediction of reference membership or adaptive stopping. We check actual counts and recall for the declared settings.

At fixed routing, widening grows the scored set, so full-score recall cannot decrease. It adds new scores and range checks, with two binary searches per cluster at each nonzero depth. A zero target reaches depth zero, whose whole-cluster range needs no binary search.

In the three-row example, C avoids one of A's three scores but performs four binary searches and builds a query key. Using the previous section's time terms, it saves time when

$$F_A a_A - F_C a_C > T_{\text{key}} + 2L_+ t_b \sum_{c \in \mathcal{O}} \log_2(n_c + 1) + T_{\text{ranges}}.$$

F_C counts C's scores; a_A and a_C include scoring and result updates. T_{key} builds the query key once, L_+ counts nonzero depths, t_b estimates comparison time, and T_{ranges} covers range bookkeeping. Whether the inequality holds depends on measured costs and the required recall.

Stopping with a score bound. There is another way to stop the same walk. After scoring a whole prefix range, we can sometimes prove that no document outside it can enter the results. Let $Q = \sum_{i=1}^d |q_i|$ be the ideal binary score, and let τ be the smallest of the K scores we have kept. At a positive depth ℓ , any unseen row must disagree with at least one of the first ℓ query

signs. Its highest possible score is therefore

$$u_\ell = Q - 2 \min_{1 \leq i \leq \ell} |q_i|, \quad \boxed{\text{stop if } \tau > u_\ell.}$$

The smallest weight appears because the best unseen row would choose the least costly sign to get wrong. If we have K retained results and this inequality holds, the answer is exact within the opened clusters.

In our three-document example, the prefix 1100* contains scores 2.5 and 1.9. The four prefix weights are 0.7, 0.5, 0.4 and 0.6, so an unseen row can score at most $2.5 - 2(0.4) = 1.7$. Both retained scores are higher. C can stop after scoring two rows, even when the candidate target is zero.

I added this as the optional `stop_when_exact` setting. With candidate target zero, the walk continues until this bound proves the answer or until it reaches depth zero. A positive candidate target can still stop approximately. The default behavior is unchanged. The code keeps a small rounding margin and does not discard score ties.

The query sum and prefix minima take $O(d + x)$ setup work for starting depth x . Each later bound check is constant work. The minima use 8x bytes and the sum uses eight bytes; the stored index does not grow. These costs are added to T_{key} and T_{ranges} in the time formula above.

This also gives us a count prediction to test. Suppose we already know the exact K th score for the opened set, calling it τ_q^* . Let y_q be the deepest stored prefix whose unseen bound is below that score, or zero if none exists. We use the same rounding margin as the code. With no candidate-count stop, a walk starting at x ends at

$$\ell_q = \min(x, y_q), \quad L_q = x - \ell_q + 1.$$

Every true top- K row must lie inside a prefix with such a bound. Scoring that range therefore produces the required score and lets the actual search stop there. Starting deeper than y_q adds lookups while reaching the same final range; starting shallower may score more rows. I use the saved reference scores to check this prediction offline. The search itself uses only the scores it has found.

3.4 Comparison

I keep the collection, embedding model and code length fixed for this comparison. For method a , let Θ_a be its tested settings and θ one choice. $T_{a,\text{measured}}$ is median whole-query time, $\bar{R}_a$ average recall, and $M_{a,\text{process}}$ the reported process-memory bound. With required recall ρ and RAM limit B_{max} in bytes, we choose the qualifying setting with the smallest time:

$$\theta_a^* = \underset{\theta \in \Theta_a}{\operatorname{argmin}} T_{a,\text{measured}}(\theta) \quad \text{subject to} \quad \bar{R}_a(\theta) \geq \rho, \quad M_{a,\text{process}}(\theta) \leq B_{\text{max}}.$$

Method	Adjustable settings	Work that may be reduced
Original method	Cluster count C and probes P	Documents reaching the scorer
Bitplanes	C , P , leaf size and node budget	Scores, at the cost of mask and branch work
Backward Walk	C , P , starting depth x , candidate target M , exact-stop option	Scores, at the cost of prefix lookups and new ranges

Here C is cluster count, P opened clusters, x starting depth and M candidate target. For each method, I choose its fastest qualifying setting using the same pool, K and recall target. We retain failed settings and repeat selected ones in fresh processes. The comparison is limited to the declared grid.

A larger collection does not automatically make B faster. The full-score term grows with the number of opened documents, but each dense mask pass grows with that number too. If B needs many alternative branches to preserve recall, those mask passes can cost more than the scores they avoid. Larger collections can also change how much pruning is possible, so we measure the work at the tested settings instead of assuming that the two latency curves must cross.

4 Experiments

4.1 Setup

To see where each method helps, I started with 256-bit codes from two collections. I then added shorter and longer codes from the same cached embeddings. Within one setup, A, B and C receive exactly the same document codes, query vectors and reference IDs. I change the index and search settings, then compare the time needed to reach the same recall target.

Collection	Embedding model	Documents	Queries	Bits
MS MARCO	Nomic v1.5	1,000,000	1,000	64, 256, 768
Quora (BEIR)	Qwen3 0.6B	522,931	1,000	32, 256, 1024

I reused the cached Nomic document embeddings and selected 1,000 queries from MS MARCO’s official `dev.small` file [6, 4]. For Quora, I encoded the full BEIR corpus and selected 1,000 official test queries [7, 5]. Both random selections were fixed before search. Corpus duplicates remain; none of the selected Quora queries has identical text or ID in its corpus.

Each model produces a full vector once. I take supported prefixes, normalize them, and store their signs. Each length has its own reference: score every document and keep the best 100, resolving ties by document ID. The first one or ten IDs give the smaller result sets. Recall here measures recovery of these binary-score references; human relevance labels are a separate measure.

The machine is an Apple M5 Max with 128 GiB of memory. I use one native search query at a time and one Faiss search thread, with the same 32 GB process limit for all methods. Reported query time starts with a cached query vector and ends with the returned IDs. It includes selecting clusters and searching their documents. Embedding generation and text reranking are outside this timer. Encoding and reference preparation finish before timed searches begin.

4.2 Parameters

For each cluster layout, I first count how many probes are needed to include the required fraction of reference IDs. A scores every document in those clusters, so it can recover all of those opened reference IDs. I give B and C the same probe choices; their local search can still miss a result, so they may need to open more clusters to reach the target.

Parameter	First sweep
Number of clusters	1, 64, 1024, 4096
Results returned	1, 10, 100
Average recall targets	50%, 80%, 90%, 95%, 99%
Opened clusters	Probe counts derived for those targets, plus every cluster
B: leaf size	128 or 2048 documents
B: visited-set budget	512 or 8192; both split nodes and leaves count
B: random exploration	0; visit the lowest-penalty waiting set first
C: starting prefix	16 bits
C: candidate target	1,000; 10,000; 100,000; half the collection; or 0

For C, target zero continues to depth zero and scores every opened document. A positive target stops after a complete depth has produced enough candidates. This can exceed the requested count when the next prefix contains many documents.

The two 256-bit grids each contain 570 settings. The wider Qwen grid keeps result counts 1 and 100; full-width Nomic focuses on 100 results at 95% and 99% recall. Across the first sweeps and follow-ups, 3,679 of 4,024 scheduled A/B/C settings qualify, plus all 72 probability settings. Time limits leave three grids partial. The saved study lists their completed and missing settings; only complete measurements enter the comparisons.

For shorter codes, I add leaves of 8 and 32 rows, probes aimed at 99.5% routing coverage, and C starting depths 0, 1, 4, 16 and 32. B also tries visiting the deeper set first when penalties tie, instead of creation order. Both branches and the score bound stay unchanged.

I build clusters from either normalized float vectors or document signs scaled by $1/\sqrt{d}$. The second option trains on the representation used by the binary scorer. All methods receive both options with unchanged codes, queries and references. The short-code follow-ups include 16, 64, 256, 1024 and 4096 clusters.

C’s exact-stop option uses target zero with starting depths 1, 4 and 32, alongside a depth-zero scan. A one-cluster Qwen32 run checks exact search over the whole collection. For B, a separate experiment holds the layout and leaf size fixed while comparing probabilities 0, 0.1 and 0.2, three seeds and two budgets. Probability changes which waiting branch we visit next; both alternatives are still kept.

Every completed setting uses all 1,000 queries, including settings that miss the recall target. I select each method’s fastest qualifying settings and repeat those choices in three fresh processes with varied run order. Small batches and saved checkpoints limit the time lost to slow settings.

If B scores 512 whole clusters without splitting, its direct alternative is A with 512 probes. I add these missing A probe counts wherever B or C appears faster, and give C its depth-zero setting at A’s selected layouts. Each method can therefore use the simpler scan when branching or widening adds no benefit.

4.3 Results

The clearest bitplane result is Quora with 32-bit Qwen codes and one returned result. After giving all methods clusters trained on the binary document vectors, B takes 0.081 ms at 99.2% recall, compared with A’s 0.195 ms and C’s 0.169 ms at 99.0%. These are repeated times for the exact settings selected in the main runs, using all 1,000 queries.

Method	Clusters	Probes	Local setting	Recall	ms	Rows scored
A	4096	180	Scan opened rows	99.00%	0.195	24,181.1
B	16	9	Leaf 128; budget 8192; deeper ties	99.20%	0.081	817.8
C	1024	78	Start 4; target 0; exact stop	99.00%	0.169	26,153.5

Rows scored is the mean per query; time is the median over the three repeated processes’ 3,000 query observations. B opens clusters containing about 295,031 rows but scores only 818 of them, while visiting about 53,693 split-mask words. Its three process medians range from 0.08025 to 0.08167 ms, below A’s 0.19223–0.19708 ms and C’s 0.16781–0.16994 ms. B’s finite budget still makes this an approximate setting: 99.2% is the observed recall, not an exactness guarantee. C’s bound proves the answer within its opened clusters; routing can still exclude a global neighbor.

If we ask for 100 results, the conclusion changes. The table below uses the same 99% recall requirement for every method and reports the repeated median of its selected setting.

522,931 documents · 1,000 queries · 32 bits

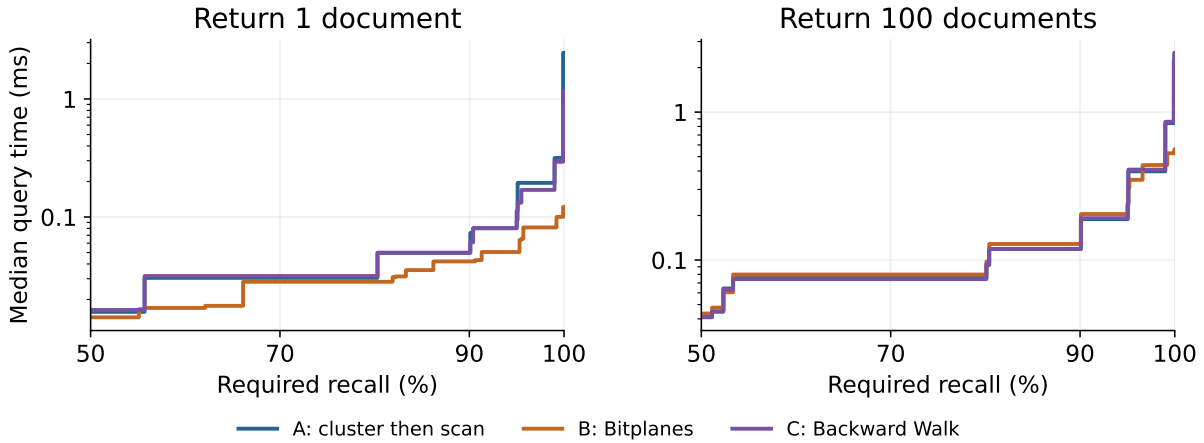


Figure 8: Qwen32 recall and query time, starting from cached vectors. The vertical axes use a log scale. The curves choose the fastest completed settings meeting each recall; the tables use their repeated timings.

Collection/model	Bits	A (ms)	B (ms)	C (ms)
MS MARCO/Nomic	64	1.915	2.071	1.875
MS MARCO/Nomic	256	13.453	13.866	14.645
MS MARCO/Nomic	768	45.088	45.217	45.196
Quora/Qwen	32	0.400	0.439	0.404
Quora/Qwen	256	3.564	3.721	4.039
Quora/Qwen	1024	22.863	23.098	22.930

Every entry reaches at least 99% recall again in its repeats. At these selected settings, B does no splitting and C starts at depth zero. Within each row, all three methods use the same 4096-cluster layout, probe count and number of document scores. C has a slightly lower median on Nomic64, but it avoids no scores there. Small timing differences between these scan-like paths do not establish a pruning advantage.

The earlier Qwen32 top-100 lead for B disappears when A also receives the binary-trained layouts. With the stronger routing, A opens 380 clusters and scores about 50,865 rows at 0.400 ms; B and C score the same rows at 0.439 and 0.404 ms. For one result, however, B’s pruning advantage remains, and C also beats the best tested A setting, although B is faster overall. This is why I compare each method with A’s own selected settings rather than only holding A at one convenient probe count.

Code length. The 32-bit result answers a different ranking question from the full 1024-value embedding. To measure that difference, I find the best full-float score over the entire corpus, then rescore each width’s exact binary top 100 using those full vectors. “Agrees” means its score is within 10^{-5} of the full-float best; equal-score documents need not have the same ID.

Model/collection	Bits	Binary top 1 agrees	Best score reached within binary top 100
Qwen/Quora	32	14.0%	73.0%
Qwen/Quora	256	75.8%	100.0%
Qwen/Quora	1024	88.6%	100.0%
Nomic/MS MARCO	64	17.6%	65.6%
Nomic/MS MARCO	256	48.0%	97.2%
Nomic/MS MARCO	768	70.2%	99.9%

For Qwen32, even the exact binary top-1 answer reaches the full-float best score on only 14% of

queries. Keeping the binary top 100 raises that to 73%, compared with 100% at 256 bits. So a faster, high-recall search over 32-bit codes does not by itself preserve full-embedding results. These are score-agreement diagnostics, not human relevance scores or measured end-to-end quality for B or C. An approximate search may also miss some of those exact binary candidates.

Exact code matches. A separate global experiment removes routing entirely and requires the exact best binary result. I select one setting per method on all 1,000 Qwen32 queries, then split those same repeated observations by whether the query’s full 32-bit sign pattern exists in the corpus. There are 82 such queries and 918 without a matching code. The settings are not retuned for either group: B uses leaf 2048 with no node limit, and C starts at 32 with target zero and the exact-stop bound.

Query group	Queries	A (μ s)	B (μ s)	C (μ s)
All queries	1000	2481.667	124.084	1163.521
Complete code present	82	2481.250	128.521	4.875
Complete code absent	918	2481.708	123.750	1170.730

All three methods return the exact binary result in every group. On the 82 matching-code queries, C scores only 1.68 rows on average and stops after its first pair of boundary searches. This is a real case where direct prefix lookup helps. A matching binary code does not imply identical text. But on the remaining 918 queries, it scores about 303,177 rows on average while widening the prefix. B therefore remains faster over the full 1,000-query batch. The 4.875 μ s result belongs to that 8.2% subgroup under the same global settings; it is not C’s overall time or a comparison with separately tuned subgroup indexes.

4.4 Checking the calculations

I checked stored field sizes and operation counts separately from the timings. Across 13 saved work-check reports, all 890 index-storage checks match the formulas, including padding at cluster boundaries; the saved query-count checks report no failures. These checks do not assign a constant time to a mask word or document score.

The score gap helps explain why short codes behave differently. For each Qwen query, I divide the gap between the ideal binary score Q and the true best score by Q . The median gap is 2.60% at 32 bits, 14.48% at 256 bits and 16.15% at 1024 bits. When a retained score is close to Q , a smaller mismatch penalty can rule out a branch. The short-code result makes such pruning possible earlier; the representation table shows the separate cost of changing the ranking problem.

At the other extreme, one completed global Nomic768 setting with $K = 100$, leaf 2048 and budget 8192 scores all one million rows. It also visits 21.57 million split-mask words and 21.46 million final-mask words per query on average: about 43.03 million word visits without avoiding a full score. More available RAM would not remove that work.

Stopping depth. For the global Qwen32 exact-stop runs, I use the saved exact best score to calculate y_q , then predict $\ell_q = \min(x, y_q)$. Counting the corresponding sorted prefix range gives the number of scored rows; the visited-depth and lookup counts follow from $x - \ell_q + 1$. All four predictions match all 3,000 query records: the same 1,000 queries at starts 1, 4 and 32.

Start x	Mean final depth	Mean rows scored	Mean visited depths	Mean boundary searches
1	0.665	353,270.1	1.335	2.000
4	1.497	290,211.1	3.503	6.336
32	4.505	278,316.6	28.495	56.320

For $K = 1$, 665 queries prove a stop above the root. Starting at 32 reduces the mean number of scores, but requires many more boundary searches. The prediction uses reference scores only in this offline check; actual search uses the scores it has found. The saved $K = 100$ predictions were not measured in this global run, so they are not included in the 3,000 matching records.

Memory at the selected settings. For the Qwen32 top-1 settings, the verified native fields plus router storage and the largest repeated-process memory peak are:

Method	Native and router fields (MB)	Repeated-process peak (MB)
A	8.924	77.971
B	10.463	72.139
C	10.598	77.529

MB here means one million bytes. The field count includes codes, IDs, bitplanes or prefix keys, and router centroids and labels. It excludes unused capacity and other process memory, so it must not be confused with the measured peak. Even the selected Nomic768 B setting uses 225.1 MB of stored fields and a 505.6 MB process peak. The common 32 GB cap does not determine these selected winners.

Random visits. I also held the Qwen32 layout at 64 float-trained clusters, opened 16, and kept leaf size 32 while varying B’s exploration probability. The following top-1 results use three seeds per probability. Recall is their mean and time is the median of their query-time medians.

Probability	Budget 128		Budget 8192	
	Recall	ms	Recall	ms
0	87.30%	0.0460	95.30%	0.0766
0.1	83.33%	0.0479	95.30%	0.0857
0.2	70.90%	0.0490	95.30%	0.0959

With the smaller budget, random visits change which branches are processed before the limit, and recall falls. With the larger budget, recall stays at 95.3% while time increases. The Nomic64 runs show no consistent time gain at matching recall either. This experiment does not establish that randomness can never help; it does not improve the tested tradeoff here.

4.5 Code and reproduction

The C++ scorer is shared by all three methods. A and B are in `cpp/index.cpp`; C is in `cpp/prefix_index_v2.cpp`. The Python worker in `experiments/prefix_batch_worker_v2.py` times routing and search, records the work counts, and compares returned IDs with the exact references.

After preparing the arrays using `experiments/prepare_prefix_pools_v2.py`, we can run one complete setup with this Python flow:

```
import json
from pathlib import Path
from experiments import prefix_study_v2 as study
path = Path("configs/prefix-study-v2/qwen-quora-d256.json")
config = json.loads(path.read_text())
output = Path("results/my-quora-run")
study.prepare(config, output) # Build clusters and the schedule.
study.run(output)             # Measure the declared settings.
study.summarize(output)        # Select by recall and time.
study.repeat(output)           # Repeat those choices three times.
```


The configurations are in `configs/prefix-study-v2/`; input hashes, per-query records, power settings and repeated times are in `results/prefix-study-2026-09-13/`. Its README links every final table and graph to the source measurements. `docs/expanded-pool-preparation.md` gives the data and encoding commands.

The root README maps the paper to its code. In particular, `experiments/prefix_comparison_v2.py` combines only runs with matching inputs; `experiments/prefix_exact_prediction_v2.py` checks prefix stopping counts; and `experiments/prefix_representation_v2.py` measures agreement with full float vectors.

5 Limitations and next steps

The timings describe this CPU implementation. All three methods reuse the same C++ lookup-table scorer, which adds entries using float64. Exa’s production system and its SIMD kernels are not reproduced here. A faster scoring kernel can change the balance between scoring documents and maintaining an index search.

I hold the collection, model and code length fixed when choosing each method’s parameters. Shorter codes change the reference ranking, so their binary recall does not establish the same retrieval quality as a longer representation. The source datasets provide human relevance labels for that separate evaluation.

These are static indexes built before querying. The experiment does not measure document updates or concurrent requests. In particular, keeping rows sorted still costs work when new documents are added. The memory formulas count the binary search index and its query buffers; an application that keeps uncompressed vectors or document text in RAM must add that storage.

Prefix expansion also appears in LSH Forest [8] and PUFFINN [9]. Those methods use multiple hash indexes and different stopping rules. Their recall guarantees do not transfer to a single prefix in the embedding’s original coordinate order.

6 Conclusion

For Qwen32 with one returned result, B takes 0.081 ms at 99.2% binary recall, versus A’s 0.195 ms and C’s 0.169 ms at 99.0%. All methods can choose binary-trained clusters. With 100 results at the 99% target, the selected B and C settings instead become scans. C has a narrower exact-search benefit: it takes 4.875 μ s on the 82 queries whose complete code exists, while B remains faster over all 1,000 queries.

These results identify useful tested settings, not a universal optimum. Qwen32’s exact binary top 1 agrees with the full-float best score on only 14% of queries, so the short-code speed does not preserve full-embedding quality. For a fixed representation and recall requirement, the decision remains whether the scores avoided pay for the added mask or prefix work. The measured counts let us check that explanation directly.

References

- [1] The Exa Team. *How we built a web-scale vector database*. 17 December 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] Y. Malkov and D. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2016; revised 2018. <https://arxiv.org/abs/1603.09320>.
- [3] A. Kusupati et al. *Matryoshka Representation Learning*. NeurIPS, 2022. <https://arxiv.org/abs/2205.13147>.

- [4] Nomic AI. *nomic-embed-text-v1.5*. Model card, accessed 13 September 2026. The saved manifest identifies the exact model revision. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.
- [5] Y. Zhang et al. *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. 2025. <https://arxiv.org/abs/2506.05176>. Encoding recipe: <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>.
- [6] P. Bajaj et al. *MS MARCO: A Human Generated Machine Reading COmprehension Dataset*. 2016; revised 2018. <https://arxiv.org/abs/1611.09268>. Passage-ranking data: <https://github.com/microsoft/MSMARCO-Passage-Ranking>.
- [7] N. Thakur et al. *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. NeurIPS Datasets and Benchmarks, 2021. <https://arxiv.org/abs/2104.08663>. Dataset catalog: <https://github.com/beir-cellar/beir>.
- [8] M. Bawa, T. Condie and P. Ganesan. *LSH Forest: Self-Tuning Indexes for Similarity Search*. WWW, 2005, pp. 651–660. <https://www.cs.princeton.edu/courses/archive/spring06/cos592/bib/LSHForest-bawa05.pdf>.
- [9] M. Aumüller, T. Christiani, R. Pagh and M. Vesterli. *PUFFINN: Parameterless and Universally Fast Finding of Nearest Neighbors*. ESA, 2019. <https://doi.org/10.4230/LIPIcs.ESA.2019.10>.